Logo

Universitas Gadjah Mada
Department of Electrical Engineering and Information Technology

---

OpenGL Project Report

Tung Tung Sahur Model Visualization with Phong Shading

Group Members

No.	Name	ID Number
1	Member Name 1	ID Number 1
2	Member Name 2	ID Number 2
3	Member Name 3	ID Number 3

1 Task Objectives

The objective of this task is to create a 3D visualization program using OpenGL that meets the three main parts of the task specification:

1. Implement vertex and fragment shaders with the Phong lighting model.
2. Create a function to load a 3D model into the scene, using VBO and VAO to process the model data on the GPU.
3. Create separate model, view, and projection matrix configurations.

The implementation plan was carried out in stages. First, the program sets up the OpenGL window, loads the `TungTungTungSahur.obj` model file, and stores the vertex position and normal data. Second, this data is sent to the GPU using VAO and VBO. Third, shaders are created to calculate the object's color using ambient, diffuse, and specular components from the Phong model. Finally, the program separates transformation calculations into model matrix, view matrix, and projection matrix functions to clarify the rendering flow.

2 Setup

The program is written in C++ and built using a `Makefile`. The libraries used are:

- **OpenGL**: The main API for 3D graphics rendering.
- **GLFW**: Used to create the window, handle the OpenGL context, and capture keyboard input.
- **GLEW**: Used to load modern OpenGL functions after the context is created.

- **GL**: The system OpenGL library linked during the build process.

The build configuration is located in the `Makefile`. The program is compiled with the C++17 standard, and GLFW/GLEW dependencies are fetched via `pkg-config`.

```
CXXFLAGS := -std=c++17 -Wall -Wextra -O2
CPPFLAGS := $(shell pkg-config --cflags glfw3 glew)
LDLIBS := $(shell pkg-config --libs glfw3 glew) -lGL -lm
```

The program can be built and run with the following commands:

```
make
make run
```

3 Mathematical Formulas Used

This section explains the mathematical formulas directly related to the program's implementation. These formulas are used for vector operations, model normal loading, camera settings, matrix transformations, and Phong lighting.

3.1 Vector Operations

The program uses the dot product to calculate the directional proximity of two vectors, such as between the surface normal and the light direction.

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z \tag{1}$$

The cross product is used to obtain face normals when the OBJ file does not provide them. If a triangle has vertices $\mathbf{p}_0$, $\mathbf{p}_1$, and $\mathbf{p}_2$, the face normal is calculated as:

$$\mathbf{n}_{face} = \frac{(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)}{\|(\mathbf{p}_1 - \mathbf{p}_0) \times (\mathbf{p}_2 - \mathbf{p}_0)\|} \tag{2}$$

Vector normalization is performed to ensure the vector length is one:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|} = \frac{\mathbf{v}}{\sqrt{v_x^2 + v_y^2 + v_z^2}} \tag{3}$$

3.2 Bounding Box, Center, and Model Scale

The OBJ loader calculates the minimum and maximum points of all vertices to create a model bounding box. The model's center point is calculated as:

$$\mathbf{c} = \frac{\mathbf{p}_{min} + \mathbf{p}_{max}}{2} \tag{4}$$

The model's maximum size is calculated from its extent:

$$\mathbf{e} = \mathbf{p}_{max} - \mathbf{p}_{min} \tag{5}$$

$$e_{max} = \max(e_x, e_y, e_z) \tag{6}$$

The model scale in the program is designed to ensure the object has an appropriate visual size in the scene:

$$s = \begin{cases} \frac{2.8}{e_{max}}, & e_{max} > 0 \\ 1, & e_{max} = 0 \end{cases} \tag{7}$$

3.3 Model Matrix

The model matrix is used to move the model to the scene's center and adjust its size. The program implementation uses translation towards the negative model center, followed by scaling:

$$M = S(s) \cdot T(-\mathbf{c}) \tag{8}$$

The vertex position in world coordinates is calculated as:

$$\mathbf{p}_{world} = M \cdot \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} \tag{9}$$

3.4 View Matrix

The view matrix is created using the `lookAt` method. Given the camera position **eye**, the target **center**, and the upward direction **up**, the camera basis is calculated as:

$$\mathbf{f} = \frac{\mathbf{center} - \mathbf{eye}}{\|\mathbf{center} - \mathbf{eye}\|} \tag{10}$$

$$\mathbf{s} = \frac{\mathbf{f} \times \mathbf{up}}{\|\mathbf{f} \times \mathbf{up}\|} \tag{11}$$

$$\mathbf{u} = \mathbf{s} \times \mathbf{f} \tag{12}$$

Based on this basis, the view matrix composes the camera orientation and translation based on the camera position:

$$V = \begin{bmatrix} s_x & u_x & -f_x & 0 \\ s_y & u_y & -f_y & 0 \\ s_z & u_z & -f_z & 0 \\ -\mathbf{s} \cdot \mathbf{eye} & -\mathbf{u} \cdot \mathbf{eye} & \mathbf{f} \cdot \mathbf{eye} & 1 \end{bmatrix} \tag{13}$$

3.5 Projection Matrix

The projection matrix uses perspective. With vertical field of view $fovy$, aspect ratio a, near plane n, and far plane f, the perspective base value is:

$$q = \frac{1}{\tan\left(\frac{fovy}{2}\right)} \tag{14}$$

The projection matrix used by the program is:

$$P = \begin{bmatrix} \frac{q}{a} & 0 & 0 & 0 \\ 0 & q & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & -1 \\ 0 & 0 & \frac{2fn}{n-f} & 0 \end{bmatrix} \tag{15}$$

The final transformation sequence in the vertex shader is:

$$\mathbf{p}_{clip} = P \cdot V \cdot M \cdot \begin{bmatrix} p_x \\ p_y \\ p_z \\ 1 \end{bmatrix} \tag{16}$$

3.6 Normal Matrix

Simply multiplying normals by the model matrix is insufficient, especially when scaling is involved. Therefore, the vertex shader uses a normal matrix:

$$N = \left(M_{3\times3}^{-1}\right)^T \tag{17}$$

The world normal is calculated as:

$$\mathbf{n}_{world} = \frac{N \cdot \mathbf{n}_{model}}{\|N \cdot \mathbf{n}_{model}\|} \tag{18}$$

3.7 Phong Lighting

Phong lighting in the fragment shader consists of ambient, diffuse, and specular components. The light direction and view direction are calculated as:

$$\mathbf{l} = \frac{\mathbf{p}_{light} - \mathbf{p}_{world}}{\|\mathbf{p}_{light} - \mathbf{p}_{world}\|} \tag{19}$$

$$\mathbf{v} = \frac{\mathbf{p}_{camera} - \mathbf{p}_{world}}{\|\mathbf{p}_{camera} - \mathbf{p}_{world}\|} \tag{20}$$

Diffuse component:

$$I_d = \max(\mathbf{n} \cdot \mathbf{l}, 0) \tag{21}$$

Light reflection direction:

$$\mathbf{r} = 2(\mathbf{n} \cdot \mathbf{l})\mathbf{n} - \mathbf{l} \tag{22}$$

Specular component:

$$I_s = \begin{cases} \max(\mathbf{v} \cdot \mathbf{r}, 0)^\alpha, & I_d > 0 \\ 0, & I_d = 0 \end{cases} \tag{23}$$

where α is the `shininess` value. The final fragment color is calculated as:

$$\mathbf{C}_{final} = \mathbf{L}_{ambient}\mathbf{M}_{ambient} + I_d\mathbf{L}_{diffuse}\mathbf{M}_{diffuse} + I_s\mathbf{L}_{specular}\mathbf{M}_{specular} + \mathbf{C}_{rim} \tag{24}$$

3.8 Height-Based Color Variation

Fragment shader experiments added color variation based on vertex height. The formula used is:

$$b = 0.5 + 0.5 \sin(18y) \tag{25}$$

The value b is used to mix the diffuse color intensity:

$$\mathbf{M}'_{diffuse} = \mathbf{M}_{diffuse} \cdot (0.78(1 - b) + 1.18b) \tag{26}$$

4 Implementation

4.1 File Structure

The main project files are:

- `src/main.cpp`: Contains the entire program code, including data structures, OBJ loader, shaders, OpenGL setup, VAO/VBO setup, input handling, and the render loop.
- `TungTungTungSahur.obj`: The 3D model file loaded into the scene.
- `Makefile`: Configuration for compilation and running the program.
- `README.md`: Brief documentation for building, running, and controls.
- `report_en.tex`: This English LaTeX report file.

4.2 Basic Data Structures

The program defines several simple data structures in `src/main.cpp`. The `Vec3` structure is used for 3D vectors, `Mat4` for 4x4 matrices, `Vertex` stores position and normal, while `Mesh` stores the list of vertices from the OBJ loading, model center, and model scale.

```
struct Vertex {
    Vec3 position;
    Vec3 normal;
};

struct Mesh {
    std::vector<Vertex> vertices;
    Vec3 center;
    float scale = 1.0f;
};
```

The `Material` structure is used to store Phong material properties: ambient, diffuse, specular, and shininess.

```
struct Material {
    Vec3 ambient;
    Vec3 diffuse;
    Vec3 specular;
```

```
    float shininess = 32.0f;
};
```

4.3 Vertex and Fragment Shader Implementation

Shaders are written directly as GLSL strings inside `main.cpp`. The mesh shader consists of `kMeshVertexShader` and `kMeshFragmentShader`. The program compiles the shaders using `compileShader()` and links them into a shader program using `makeProgram()`.

The vertex shader receives two main attributes from the VAO: vertex position and vertex normal.

```
layout (location = 0) in vec3 aPosition;
layout (location = 1) in vec3 aNormal;
```

The vertex shader transforms the position from local model coordinates to world coordinates using `uModel`, then to clip space using `uView` and `uProjection`.

```
vec4 world = uModel * vec4(aPosition, 1.0);
vWorldPos = world.xyz;
gl_Position = uProjection * uView * world;
```

Vertex normals are calculated using the normal matrix:

```
vNormal = transpose(inverse(mat3(uModel))) * aNormal;
```

The normal matrix is used to keep normal directions correct when the model undergoes transformations like scaling. The results from the vertex shader, namely world position, normal, and vertex height, are sent to the fragment shader.

4.4 Phong Model Implementation

The fragment shader applies the Phong lighting model. Inside the shader, there are two GLSL structures: `Material` and `Light`. The `Material` structure contains object surface characteristics, while the `Light` structure contains the light's position and color.

```
struct Material {
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
    float shininess;
};

struct Light {
    vec3 position;
    vec3 ambient;
    vec3 diffuse;
    vec3 specular;
};
```

The diffuse component is calculated from the dot product of the surface normal and the light direction. If the surface faces the light, the diffuse value increases.

```
float diff = max(dot(n, lightDir), 0.0);
```

The specular component is calculated from the camera's view direction and the light's reflection direction. The `shininess` value determines how sharp the highlight is on the surface.

```
vec3 reflectDir = reflect(-lightDir, n);
float spec = diff > 0.0
    ? pow(max(dot(viewDir, reflectDir), 0.0), uMaterial.shininess)
    : 0.0;
```

The object's final color is a combination of ambient, diffuse, specular, and a slight static rim light to make the model's shape more visible.

```
vec3 color = ambient + diffuse + specular + rimLight;
fragColor = vec4(color, 1.0);
```

4.5 Shader and Material Experiments

To meet the vertex and fragment shader experiment requirements, the fragment shader is given color variation based on vertex height. This variation is static, keeping the scene stable.

```
float bands = 0.5 + 0.5 * sin(vHeight * 18.0);
vec3 patternedDiffuse = uMaterial.diffuse * mix(0.78, 1.18, bands);
```

The program also provides four material presets in `kMaterials`. Materials can be switched while the program is running using keys `1` through `4`. Each material has different ambient, diffuse, specular, and shininess values, changing the object's appearance from wood, gold, cyan, to metallic gray.

```
static const std::vector<Material> kMaterials = {
    {{0.18f, 0.08f, 0.03f}, {0.72f, 0.36f, 0.13f}, {0.35f, 0.22f, 0.12f}, 24.0f
        },
    {{0.10f, 0.08f, 0.05f}, {0.78f, 0.57f, 0.24f}, {0.95f, 0.82f, 0.45f}, 72.0f
        },
    {{0.02f, 0.08f, 0.09f}, {0.05f, 0.55f, 0.62f}, {0.55f, 0.90f, 0.95f}, 96.0f
        },
    {{0.04f, 0.04f, 0.045f}, {0.28f, 0.30f, 0.32f}, {0.85f, 0.88f, 0.90f},
        128.0f},
};
```

Sending materials to the shader is done via the `setMaterialUniforms()` function.

```
static void setMaterialUniforms(GLuint program, const Material& material) {
    setVec3Uniform(program, "uMaterial.ambient", material.ambient);
    setVec3Uniform(program, "uMaterial.diffuse", material.diffuse);
    setVec3Uniform(program, "uMaterial.specular", material.specular);
    glUniform1f(glGetUniformLocation(program, "uMaterial.shininess"),
        material.shininess);
}
```

4.6 3D Model Loading Function

The 3D model is loaded using the `loadObj()` function. This function opens the `TungTungTungSahur.obj` file, reads it line by line, and processes OBJ tags like `v`, `vn`, and `f`.

- `v`: Vertex position.
- `vn`: Vertex normal.
- `f`: Face or model side.

Vertex and normal indices are processed by `parseObjIndex()` and `parseNormalIndex()`. Faces with more than three vertices are broken down into multiple triangles using the triangle fan method.

```
for (std::size_t i = 1; i + 1 < face.size(); ++i) {
    std::string tri[3] = {face[0], face[i], face[i + 1]};
    Vertex out[3];
    ...
}
```

If the model does not provide normals for a face, the program calculates the normal using the cross product.

```
Vec3 faceNormal = normalize(cross(
    out[1].position - out[0].position,
    out[2].position - out[0].position
));
```

In addition to creating the vertex list, the loader also calculates the model's bounding box. The bounding box is used to find the model's center and scale so that the object appears in the center of the scene with the correct size.

```
mesh.center = (minP + maxP) * 0.5f;
Vec3 extent = maxP - minP;
float maxExtent = std::max(extent.x, std::max(extent.y, extent.z));
mesh.scale = maxExtent > 0.0f ? 2.8f / maxExtent : 1.0f;
```

4.7 VAO and VBO Usage

After the model is loaded, the vertices from `mesh.vertices` are sent to the GPU. The program creates a VAO to store the vertex attribute configuration and a VBO to store vertex data on the GPU.

```
glGenVertexArrays(1, &meshVao);
glGenBuffers(1, &meshVbo);
glBindVertexArray(meshVao);
glBindBuffer(GL_ARRAY_BUFFER, meshVbo);
glBufferData(GL_ARRAY_BUFFER,
    static_cast<GLsizeiptr>(mesh.vertices.size() * sizeof(Vertex)),
    mesh.vertices.data(),
    GL_STATIC_DRAW);
```

Position attributes are linked to location 0, while normals are linked to location 1. Both match the input declarations in the vertex shader.

```
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), reinterpret_cast<void*>(offsetof(Vertex, position)));
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,
    sizeof(Vertex), reinterpret_cast<void*>(offsetof(Vertex, normal)));
glEnableVertexAttribArray(1);
```

In the render loop, the mesh is drawn using `glDrawArrays()`.

```
glBindVertexArray(meshVao);
glDrawArrays(GL_TRIANGLES, 0, static_cast<GLsizei>(mesh.vertices.size()));
```

4.8 Model, View, and Projection Matrix Setup

The program separates transformations into three functions. This separation makes the graphics pipeline clearer: the model matrix manages the object, the view matrix manages the camera, and the projection matrix manages perspective.

4.8.1 Model Matrix

The model matrix is created by `makeModelMatrix()`. This function moves the model to the scene center and applies scaling from the bounding box calculation.

```
static Mat4 makeModelMatrix(const Mesh& mesh) {
    return multiply(scale(mesh.scale), translate(mesh.center * -1.0f));
}
```

4.8.2 View Matrix

The view matrix is created from the camera position and target. The `makeViewMatrix()` function calls `lookAt()` with an upward direction of `(0, 1, 0)`.

```
static Mat4 makeViewMatrix(Vec3 camera, Vec3 target) {
    return lookAt(camera, target, {0.0f, 1.0f, 0.0f});
}
```

The camera can be controlled via the keyboard. `A`/`D` keys rotate the camera, `W`/`S` adjust the camera's tilt, and `Q`/`E` adjust the camera's distance from the object.

4.8.3 Projection Matrix

The projection matrix is created by `makeProjectionMatrix()`. This function uses perspective projection with a 45-degree field of view. The aspect ratio is calculated from the framebuffer size to ensure the display is not distorted when the window size changes.

```
static Mat4 makeProjectionMatrix(int width, int height) {
    float aspect = width > 0 && height > 0
        ? static_cast<float>(width) / static_cast<float>(height)
        : 1.0f;
```

```
    return perspective(45.0f * 3.14159265f / 180.0f,
        aspect, 0.05f, 100.0f);
}
```

The three matrices are sent to the shader before the object is drawn.

```
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uModel"),
    1, GL_FALSE, model.m);
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uView"),
    1, GL_FALSE, view.m);
glUniformMatrix4fv(glGetUniformLocation(meshProgram, "uProjection"),
    1, GL_FALSE, projection.m);
```

4.9 Render Loop and Input

The render loop runs as long as the window is not closed. Each frame, the program reads input, gets the framebuffer size, calculates the camera position, creates the model/view/projection matrices, clears the screen, and then draws the ground grid and the model. The ground grid is static, without animation. The object is also stable, without periodic beat or scaling effects.

Materials are selected using keys `1` through `4`.

```
for (int i = 0; i < static_cast<int>(kMaterials.size()); ++i) {
    if (glfwGetKey(window, GLFW_KEY_1 + i) == GLFW_PRESS) {
        materialIndex = i;
    }
}
```

5 Screenshots

This section is prepared for screenshots of the program running in various configurations. Screenshots will be filled from the results of the locally running program. Run the program with:

```
make run
```

Then press keys `1`, `2`, `3`, and `4` to take screenshots of different materials. Save the screenshot files to the `screenshots/` folder with the following names.

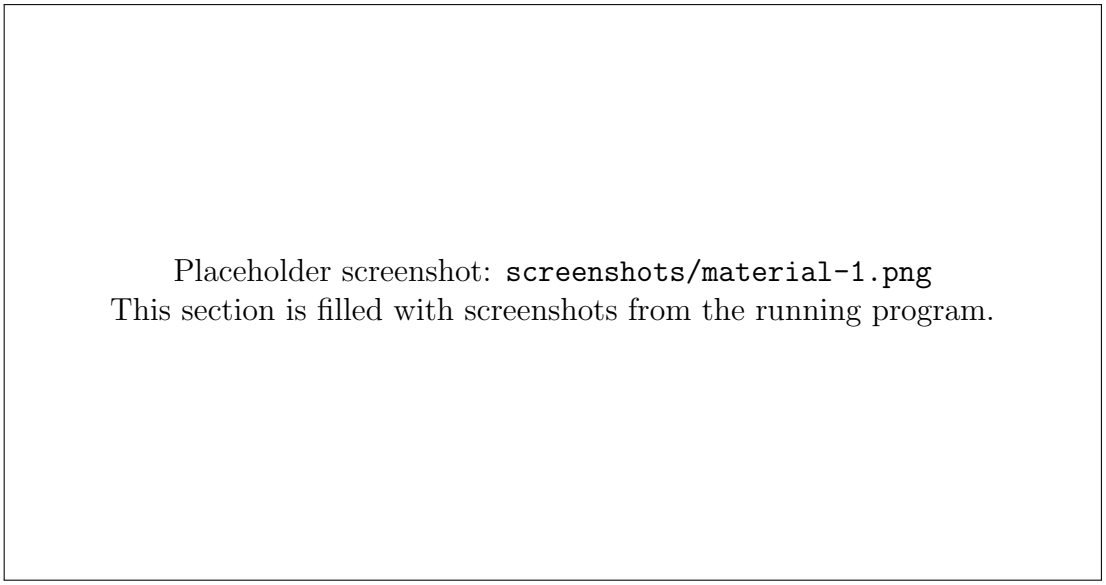

Figure 1: Program running with material preset 1.

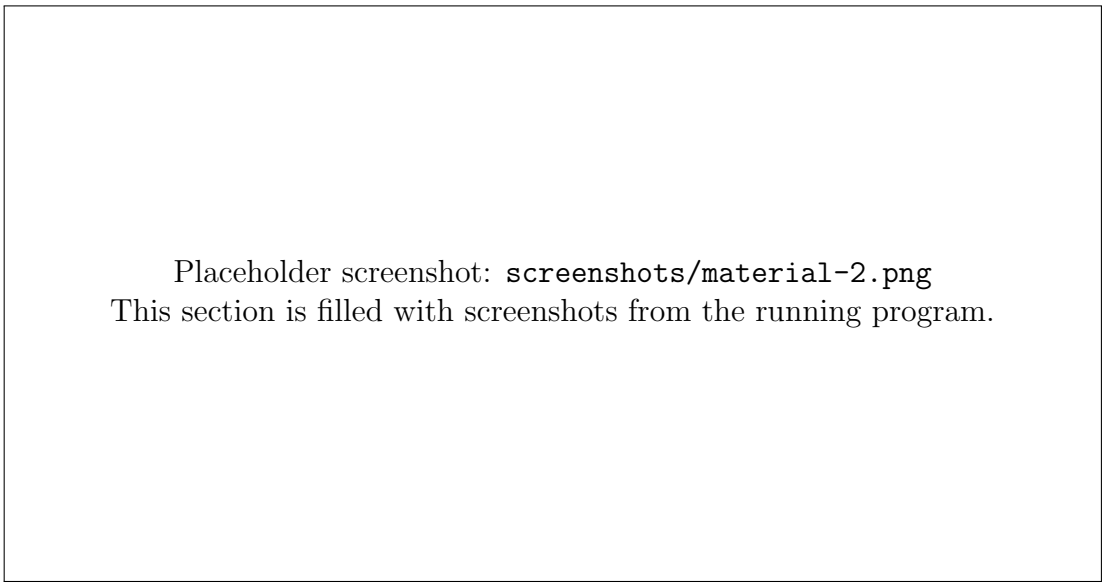

Figure 2: Program running with material preset 2.

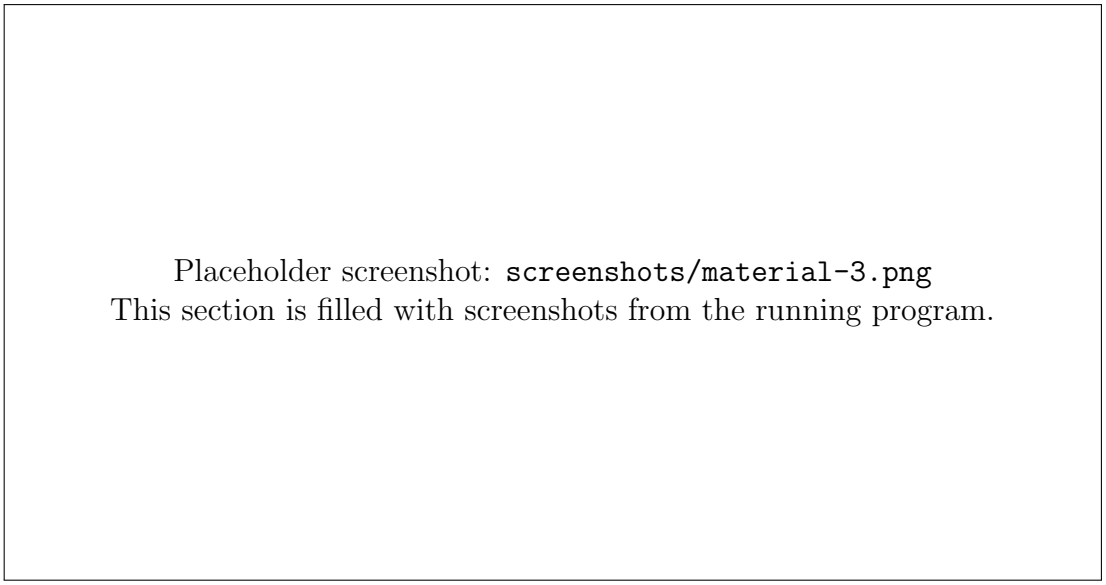

Figure 3: Program running with material preset 3.

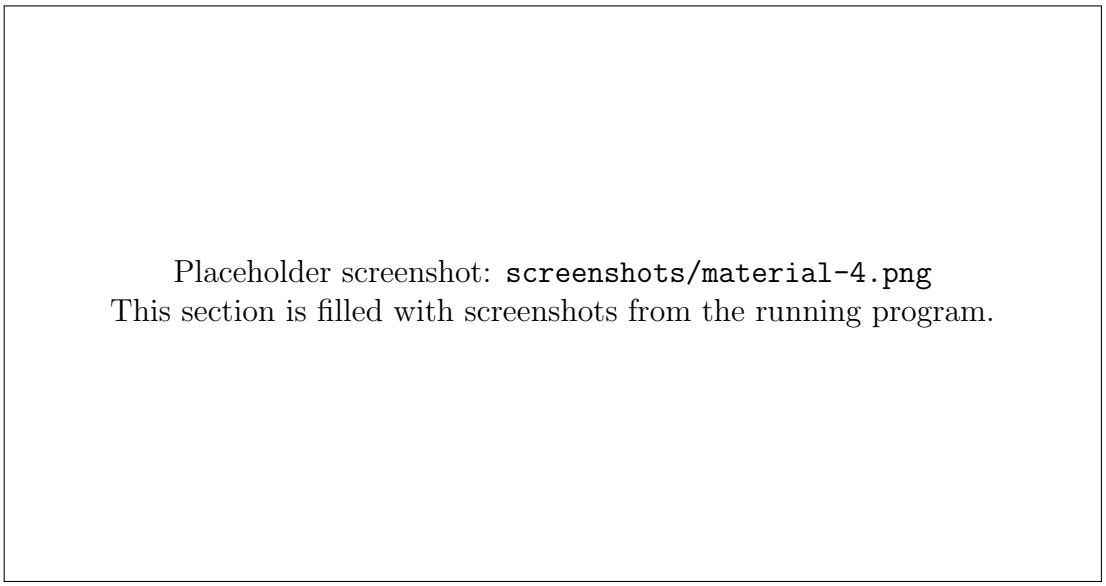

Figure 4: Program running with material preset 4.

6 Conclusion

The program has met the three main requirements of the task. Vertex and fragment shaders have been created with the Phong lighting model. The 3D object is loaded from an OBJ file and sent to the GPU using VAO and VBO. Scene transformations have also been separated into model, view, and projection matrices. Additionally, the program provides several materials that can be changed at runtime, allowing for object appearance configurations to be compared via screenshots.